

HS4002: Lecture 1

Course Introduction

Ma Xiangyu

August 12, 2026



Introductions

Introduce yourselves!

1. Name
2. Major and year of study
3. What about this course are you most afraid of or most looking forward to?
4. If you could measure one thing about NTU life with a quantitative survey, what would it be?

Overview

Course description

- This is a practical course in which you will learn to use quantitative techniques to answer research questions.
- You (and I) will learn how to pose research questions that are interesting, important, and tractable.
- We will learn quantitative methods that can provide reasonable answers to our questions.
- We will also learn to work with contemporary tools and workflows, including AI tools, Python, version control, and modern IDEs and CLIs.

- In this class, we will learn “quantitative methods” through the lens of survey research.
- Whenever a problem “scales up,” it demands a different tool.
 - Think about how you would approach interviewing 10, 100, or 1,000 people.
- There are advantages and disadvantages to quantitative methods, of course.

- There is nothing special about “quantitative methods.” You do not need any special aptitude for mathematics or coding.

For every scientist, whether researcher, technologist, or teacher, of whatever competence in mathematics, there exists a discipline in science for which that level of mathematical competence is enough to achieve excellence. (Wilson 2013)

What you can expect to learn

- Become comfortable working with code and numbers in Python.
- Become familiar with basic quantitative research designs.
- Become statistics-literate.
- Learn to work in contemporary programming environments (Jupyter, IDEs, CLIs).
- Become comfortable working with quantitative methods in future projects.

Course Policies

How to contact me

- The best way to reach me is by email: xy.ma@ntu.edu.sg
- You can generally expect a response within 24 hours during the week, and by Monday morning if you email over the weekend.

- Office hours are scheduled hours when I will be in the office to answer any class-related questions you might have.
- Thursdays, 10.30 a.m.–12.30 p.m. (SHHK 05-40)
- No appointment needed — just stop by. I'd encourage you to drop by at least *once*.
- If you can't make that window, email me with three possible times.

Lateness policy

- You are entitled to one 24-hour extension on any class assignment with no penalty.
 - You may use this at any time at your own discretion.
 - You may use this for group components (i.e. your group is late, but YOU are exempt from penalty).
- Thereafter, late submissions will be penalized according to the following criteria:
 - One letter grade (i.e. A → B) if one day late.
 - Two letter grades (i.e. A → C) if two to three days late.
 - A grade of 0 if more than three days late.
- Negotiation around deadlines is only possible under exceptional circumstances.

Expectations

A social contract around expectations

1. You are expected to come prepared for each class having read the required readings for the day.
2. Use computers courteously.
3. Be daring but kind in the classroom.
4. Attendance is assumed; you are responsible for yourself. This applies especially to in-class tests and presentations.
5. Take academic integrity seriously.

Details on Assessments

Table 1: Assessment Breakdown

Component	Type	%
Weekly Classroom Facilitation	Group*	10%
Team Research Paper	Group*	40%
In-class Tests	Individual	40%
Class Participation	Individual	10%

Weekly Classroom Facilitation

- Teams of no more than five.
 - You will be in this group for both the classroom facilitation and team research paper.
- You'll facilitate a 30-minute classroom discussion based on a chapter from Dillman et al. (2014).
 - Weeks 5, 6, 7, 9, 10, and 11
- Open-ended!
 - E.g. a 10-minute lecture, 10-minute small-group discussion, and 10-minute whole-class discussion
- Specific due dates are listed in the syllabus.

- Group-based (same group as before).
- A semester-long project split into five components:
 1. Scoping memo (due week 4)
 2. Group presentation #1 (week 8)
 3. Group presentation #2 (week 12)
 4. Research paper + code on a public GitHub repo (week 14)
 5. Peer evaluations of contributions (week 14)

On forming teams

- At the end of week 1, I will randomly assign all of you to groups.
 - You may swap groups if you wish.
 - To change groups, arrange a swap with someone in another group and email me, copying that person.
 - Groups must be finalized by the end of week 2.

- You will be assessed on your ability to identify appropriate statistics and methods for an empirical problem and to interpret results meaningfully, including Python output.
- The first in-class test (week 7) will cover material from weeks 2 through 6.
- The second in-class test (week 13) will be cumulative, with an emphasis on material from weeks 8 through 12.

Class Participation

- Come prepared for class.
- What does it mean to “participate”?
 - Ask questions.
 - Listen and build on classmates’ perspectives.
 - Assert your own views.
- Participate actively in coding sessions!
- Lab notebook submissions also count toward participation.

- Attribute LLM use where necessary. Never take credit for what you didn't do.
- We will work with contemporary AI tools in this class (e.g. Cursor, Claude Code).
 - Use them when you run into programming problems — productively.
 - Don't expect them always to be right. Relying on them for things you can't debug yourself or don't understand is a *really bad* idea.

What to do by week 2

To do

- Finalize your groups.
 - Finalize any swaps by the end of week 2 (Sunday, 10pm).
- Once you've finalized your groups, email me your preferences for weekly facilitation.
 - Rank the weeks from most to least preferred. For example, {11, 6, . . . , 5} means that 11 is most preferred and 5 is least preferred.
 - Available weeks: 5, 6, 7, 9, 10, 11.
 - One person emails me. CC everyone else in the group on the email.
- Complete the ERIC / CITI research ethics certificate, then email me the certificate.
 - **NTU IRB minimum training requirements**

Setting up

Download and install Python

- We will use Python throughout the semester (on Windows, macOS, or Linux).
- Recommended starting points:
 - Install a recent version of Python 3 from <https://www.python.org/downloads/>.
- We will work in Jupyter notebooks (.ipynb) to begin.

Installing Git

- Git is how we will collaborate (and how your team paper lands on GitHub).
- **macOS:** Open **Terminal**, then type `xcode-select --install`.
- **Windows:** Install *Git for Windows* from <https://git-scm.com/download/win> (or `winget install --id Git.Git -e`)
 - You may use **Git Bash**, **PowerShell**, or **Command Prompt**.
 - Accept defaults; leave “Git from the command line ...” enabled.
- Check: open a new terminal → `git --version`

Making a folder for HS4002

- Pick a home for the course — somewhere you can easily find again (e.g. Documents).
- Go to Documents using your preferred terminal:
 - macOS/Git Bash: `cd ~/Documents`
 - PowerShell: `cd $HOME\Documents`
 - Command Prompt: `cd /d %USERPROFILE%\Documents`
- Then create and enter the course folder:
 - `mkdir hs4002_ay26`
 - `cd hs4002_ay26`
- Check where you are: `pwd` (macOS, Git Bash, or PowerShell) or `cd` (Command Prompt).

- A *virtual environment* (`venv`) is an isolated Python install for one project.
- Without one, packages pile into a global Python — versions collide, projects break each other, and “it works on my machine” gets worse.
- Rule of thumb: **one project, one venv**.
- We will use the standard library module `venv` (no extra install needed once Python is installed).

Creating a virtual environment

1. Open a terminal in your project folder.
2. Create the environment (once per project):
 - macOS: `python3 -m venv .venv`
 - Windows (any terminal): `python -m venv .venv`
3. This creates a `.venv` folder containing an isolated Python environment.

Activating a virtual environment

1. Activate it at the start of every new terminal session:
 - macOS: `source .venv/bin/activate`
 - Windows (Git Bash): `source .venv/Scripts/activate`
 - Windows (PowerShell):
`.\.venv\Scripts\Activate.ps1`
 - Windows (Command Prompt):
`.venv\Scripts\activate.bat`
2. If PowerShell blocks activation, first run
`Set-ExecutionPolicy -Scope Process`
`-ExecutionPolicy Bypass.`
3. Your prompt should now show `(.venv)`.

Installing packages

1. With the virtual environment active, install the required packages:

- `python -m pip install --upgrade pip`

- `python -m pip install pandas plotnine jupyter ipykernel`

2. Register it so Jupyter notebook can find it.

- `python -m ipykernel install --user --name=hs4002 --display-name="Python (hs4002)"`

Virtual environments: best practices

- Name it `.venv` (or `venv`) and keep it *inside* the project folder.
- Always activate before installing or running code.
 - Check with `which python` (macOS/Git Bash) or `where.exe python` (PowerShell/Command Prompt).

Technical Demonstration

Figure 1: Using Python to look at some *Love Is Blind* statistics



Exercise

1. Download `lib_toy.csv` and the Week 1 Jupyter notebook from NTULearn.
2. Move both into your course folder (e.g. `~/Documents/hs4002_ay26/labs`).
3. Open the notebook in JupyterLab, VS Code, or Cursor.
4. Select the Python (hs4002) kernel.
5. Run the first cells: confirm imports work and that you can load the CSV.

End.